CAD for VLSI Design
Programming Assignment 3

Graph-based STA

Advisor: Yu-Guang Chen

Student Info: 電機 4A

Student ID: 110501521

Student Name: 徐松廷

1. How to Compile and Execute My Program

a. 繳交檔結構

```
110501521_PA3/
├── 110501521_PA3.cpp
├── Makefile
├── 110501521_PA3_report.pdf
├── inc/
│   ├── liberty_parser.h
│   └── RTL_parser.h
└── src/
    ├── liberty_parser.cpp
    └── RTL_parser.cpp
```

這是 PA3 的結構，因為這次要處理的文件較多，所以我開始練習拆分檔案，根目錄下有主程式 110501521_PA3.cpp、編譯規則 Makefile 和報告檔 110501521_PA3_report.pdf。inc/資料夾放置標頭檔 liberty_parser.h 與 RTL_parser.h，定義了各自對應的功能接口；src/資料夾則包含 liberty_parser.cpp 與 RTL_parser.cpp，實作了對應的標頭檔內容。整體結構清楚區分了主程式、標頭檔、來源檔與文件。

b. My Makefile :依照這次 PA 要求，一樣要使用 Makefile 進行編譯

```
CXX = g++
CXXFLAGS = -Wall -std=c++11 -Iinc

TARGET = 110501521_PA3
SRCS = 110501521_PA3.cpp src/liberty_parser.cpp src/RTL_parser.cpp

OBJS = $(SRCS:.cpp=.o)

all: $(TARGET)

$(TARGET): $(SRCS)
	$(CXX) $(CXXFLAGS) -o $(TARGET) $(SRCS)

clean:
	rm -f $(TARGET) *.o src/*.o
```

依照這次 PA 要求，一樣要使用 Makefile 進行編譯。CXX = g++設定使用 g++作為 C++編譯器，CXXFLAGS 加上編譯選項，包括所有警告（-Wall）、使用 C++11 標準以及指定頭檔目錄 inc（-Iinc）。TARGET 指定最後生成的執行檔名稱是 110501521_PA3，SRCS 列出所有需要編譯的原始碼檔案。OBJS 本來是要列出物件檔，但實際上在這個 Makefile 中沒有用到。all 是預設目標，會呼叫$(TARGET)，也就是把所有 SRCS 一起編譯成 TARGET。$(TARGET): $(SRCS)定義了編譯的指令，用 g++把原始碼編譯成可執行檔。

c. Compiling and Executing Steps:

依照助教要求，本次 PA3 會需要按照以下方式，利用 makefile 去編譯.cpp file 並執行.exe file，產生.out 後利用助教提供的 checker 確認我們的 STA 結果是否正確:

I. Step 1: make all

II. Step 2: ./110501521_PA3 netlist.v test_lib.lib

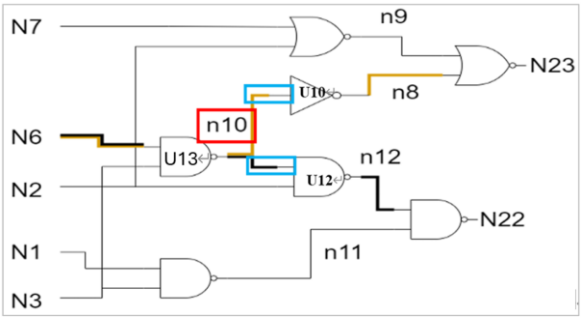
III. Step 3: ./PA3_checker case.v test_lib.lib Load_110501521_case.txt Delay_110501521_case.txt Path_110501521_case.txt

```
[s110501521@ncuee 110501521_PA3]$ ./PA3_checker c17.v test_lib.lib Load_110501521_c17.txt Delay_110501521_c17.txt Path
_110501521_c17.txt
Step 1 Correct!!!
Step 2 Correct!!!
Step 3-1 Correct!!!
Step 3-2 Correct!!!
Score: 8
```

2. The Completion of The Assignment

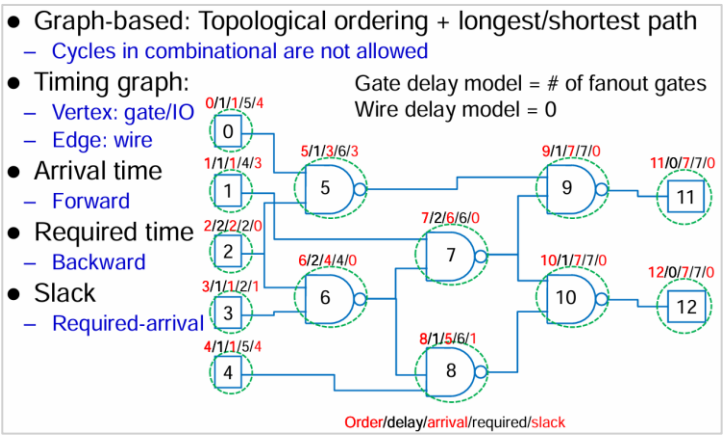
a. Stage 1: Calculate output loading

依照這次 PA3 要求，要讀取 netlist.v 以及 test_lib.lib 以實現計算每個 gate 的負載電容計算，且設定 primary output 的電容為 0.03F，步驟如下，每個 gate 都會記錄自計的 output wire，每個 wire 又同時會記錄自己是哪個 gate 的 A1 或是 A2，這樣就能計算負載電容，例如以下範例 U13 的負載電容就是 U10 的 A1 input capacitance+U12 的 A1 input capacitance

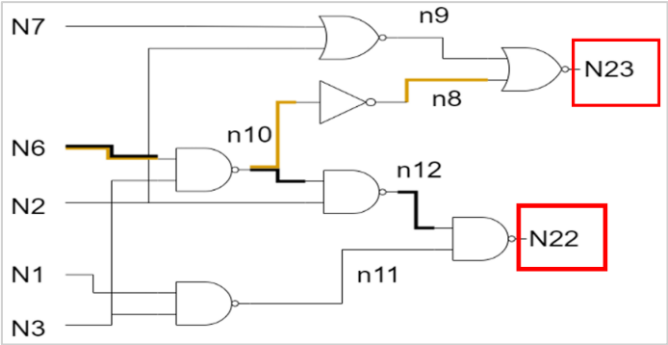


b. Stage 2 propagation delay and output transition time

這次的 PA3 主要目標是找出每一個 gate 的 output transition time 以 propagation delay。由於 gate 的特性，它必須等到所有輸入訊號都**準備好、且到達後**，才能確定該 gate 的開始運作時間。而這個開始時間再加上該 gate 本身的延遲，就能推算出訊號完整輸出的時間，因此這次的 PA3 需要反覆運用 topological ordering 來完成整體的時間推算與計算工作，例如江蕙如教授在 invited talk 說明的範例，**gate_9_start time = max (gate 5_start_time+delay, gate 7_start_time+delay)**，依照定義即可根據 topological ordering 推算所有 gate 的 start time、delay，並用查表推算 output transition time:



c. Step 3 Find the longest and the shortest delay from all output paths



這次在步驟三中，尋找 longest path 以及 shortest path 的部分需要特別留意。這裡所求的 longest path 和 shortest path 與我們在資料結構或演算法課中，使用 Dijkstra 演算法或動態規劃（dynamic programming）等方法的做法並不完全相同。**一般而言，在 graph 中，edge 本身就代表固定的 delay，但在這次的 PA3 中，邊僅僅代表依賴關係（dependency）**，真正的延遲必須依據每個 node 的輸入邊，經過推算後才能確定。

由於我們需要尋找 longest path 和 shortest path，正確的做法根據步驟二已經推算出來的所有 wire 與 net 的 start time 和 delay，進一步判斷。我們會從所有的 primary output net 中分別找出最晚完成的 primary output net (對應到 longest path)以及最早完成的 primary output net (對應到 shortest path)。接著從這些 primary output nets 出發，沿路反推回整條路徑上所經過的 nets。在反推的過程中，每一個 gate 所選擇的輸入 net，都是以該 gate 所有輸入中，以最晚到達的 net 為依據，進而決定這條 path 的走向。

3. Method of My Design - Data Structure

為了實現 step 1、step 2、step 3 的計算流程，我最上層的資料結構設計如下:

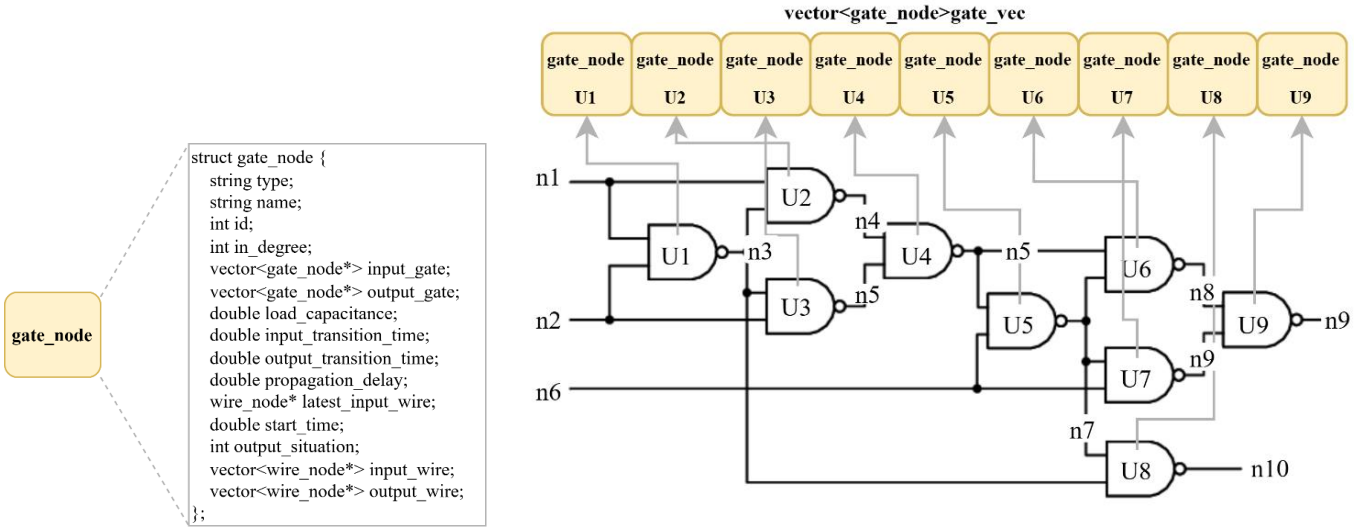
```
struct RTL_obj{
    vector<gate_node*> gate_vec;              //記錄所有gate
    unordered_map<string,wire_node*> net_map;//記錄所有wire
    vector<wire_node*> input_net;             //記錄所有input net
    vector<wire_node*> wire_net;              //記錄所有wire net
    vector<wire_node*> output_net;            //記錄所有output net
    vector<wire_node*> longest_path;          //紀錄longest_path
    vector<wire_node*> shortest_path;         //紀錄shortest_path
};
```

儲存 Liberty File 的結構如下:

```
struct liberty_file {
    vector<double> index_1;     //output_net_capacitance
    vector<double> index_2;     //input_transition_time
    struct gate_info NOR;       //NOR Gate的所有Table
    struct gate_info INV;       //INV Gate的所有Table
    struct gate_info NAND;      //NAND Gate的所有Table
};
```

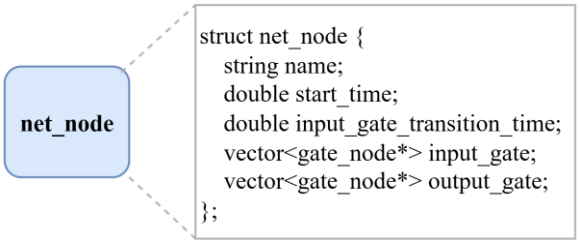
以下詳細介紹這些資料結構的定義與用途:

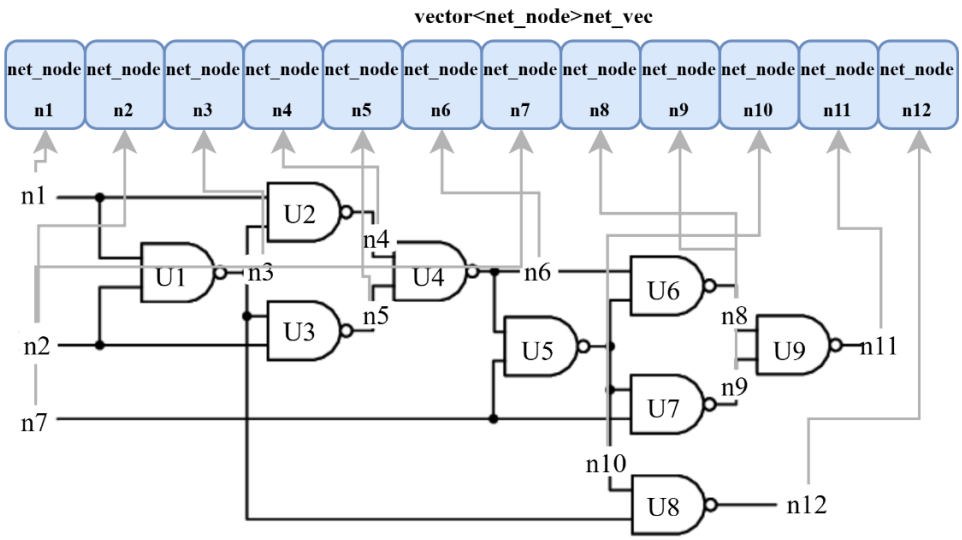
a. gate_node 與 gate_vector



在 gate_node 中，每個 gate 會有型別(如 NAND、NOR、INV)、名稱、編號等基本資訊，in_degree 及 input_gate、output_gate 用來支援步驟二的拓撲排序(Topological Sorting)。負載電容(load_capacitance)和輸出轉換時間(output_transition_time)是在步驟一計算時需要用到的參數。輸入轉換時間(input_transition_time)與傳播延遲(propagation_delay)則與步驟二的延遲計算有關。latest_input_wire 紀錄輸入中到達最晚的 wire，方便步驟三反推最長或最短路徑。start_time 表示訊號抵達 gate 的時間點(尚未加上自身延遲)，而 output_situation 則標示輸出是上升沿還是下降沿。input_wire 和 output_wire 紀錄這個 gate 的輸入和輸出連接線。而這邊會選用 vector 儲存所有 gate 的 node 的原因是因為我們不會需要對 gate 做查找，只需要遍歷，所以 vector 是最好的選擇。

b. wire_node 與 net_map

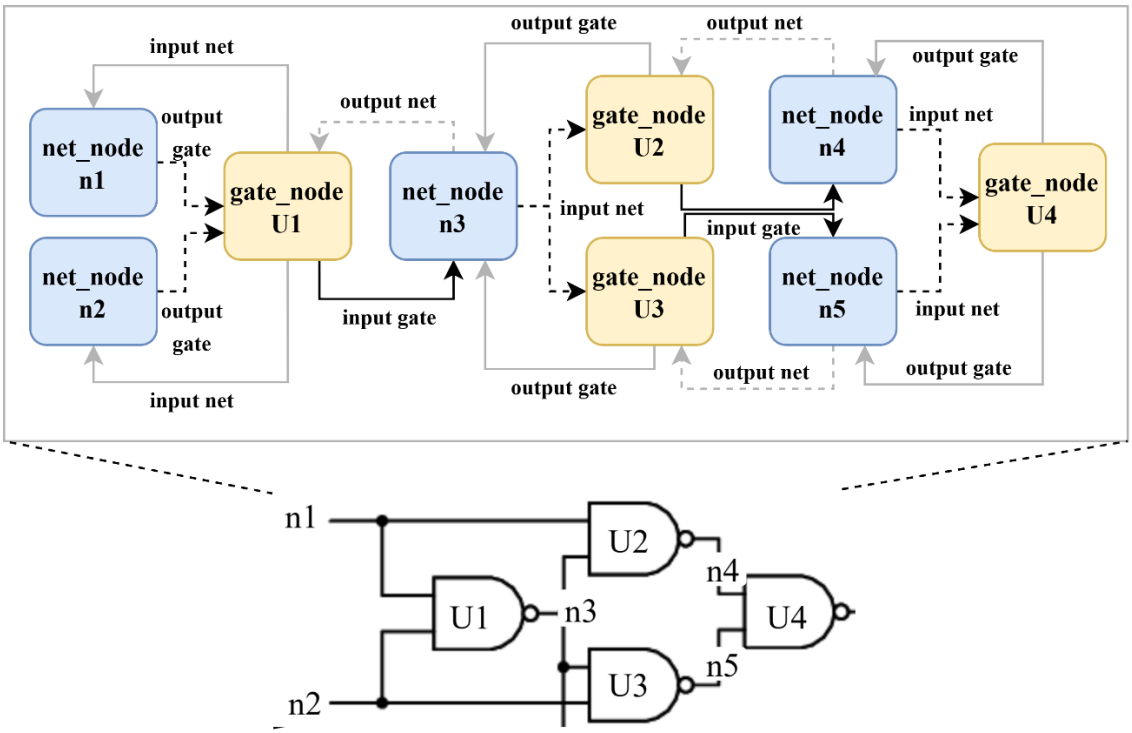




在 wire_node 中記錄所有 wire 有一個名稱（name），用來標識它的身份。start_time 表示訊號到達這條 wire 的時間（尚未加上自身延遲），是時序分析時重要資訊。

input_gate_transition_time 則用來記錄從輸入 gate 帶來的 transition time，在步驟二的延遲與 transition 計算時會使用。input_gate 代表所有驅動這條 wire 的 gate，output_gate 則代表所有接收這條 wire 的 gate，這兩個欄位在步驟二（延遲推算）與步驟三（尋找最長/最短路徑）中都扮演關鍵角色。

依照我們前面定義的資料型態(pointer)，可以將 gate_node 與 wire_node 建構出以下的 dependency，以利後續的計算:



c. input_net_vector、output_net_vector、wire_net_vector

```
vector<wire_node*> input_net;
vector<wire_node*> wire_net;
vector<wire_node*> output_net;
```

在讀取 netlist.v 時，會記錄 input、output、wire 的所有 wire 的 node，為了後續查找方便。

d. longest_path_vector、shortest_path_vector

```
vector<wire_node*> longest_path;
vector<wire_node*> shortest_path;
```

Step 3 的 path 結果會放在以下兩個 vector，會記錄 longest_path、shortest)path 的所有 wire 的 node，為了寫檔時方便。

e. Gate Table

```
struct gate_info {
    double A1_capcitance;
    double A2_capcitance;
    vector<vector<double>> cell_rise_TB;        //cell_rise Table
    vector<vector<double>> cell_fall_TB;        //cell_fall Table
    vector<vector<double>> rise_transision_TB;  //rise_transision Table
    vector<vector<double>> fall_transision_TB;  //fall_transision Table
};
```

gate_info 結構（struct gate_info）用來描述一個邏輯閘的特性。它包含兩個輸入腳位的電容 (A1_capcitance 和 A2_capcitance)，以及四個二維陣列（vector<vector<double>>）型態的查詢表（Table），分別記錄: cell_rise_TB、cell_fall_TB、rise_transision_TB、fall_transision_TB。

```
struct liberty_file {
    vector<double> index_1;     //output_net_capacitance
    vector<double> index_2;     //input_transition_time
    struct gate_info NOR;       //NOR Gate的所有Table
    struct gate_info INV;       //INV Gate的所有Table
    struct gate_info NAND;      //NAND Gate的所有Table
};
```

這個 liberty_file 結構（struct liberty_file）是用來描述整個邏輯庫（Liberty Library）中重要的時序資訊，liberty_file 把時序分析需要的索引和三種 gate 的詳細特性組合在一起，方便後續做 timing simulation 或 delay estimation。

I. **Windex_1**: 一維向量，對應 output net capacitance 的各種取樣點。

II. **index_2**: 一維向量，對應 input transition time 的各種取樣點。

III. **NOR、INV、NAND**: 三個 gate_info 結構，分別儲存 NOR、INV 和 NAND 三種邏輯閘的時序表資料，包括延遲與轉換時間表。

4. Method of My Design - Algorithm

a. 雙線性內插公式

在 step 2 中需要使用雙線性內插尋找 cell_rise、cell_fall、rise_transition、fall_transition 的數值，我們會先根據該 gate 的 input transition time (x)和 load capacitance (y)選定 var 1(x)與 var 2(y)的 interval，這樣即可決定 x0、x1、y0、y1，並使用以下公式求得最後內差的數值，**如果 x 或是 y 沒有落在表中所提供的任何 interval 中，<u>就選表中最接近的 interval 進行內插即可</u>:**

$$f(x,y) = (1-t)(1-u)f_{00} + t(1-u)f_{10} + (1-t)uf_{01} + tuf_{11}$$

$$t = \frac{x - x_0}{x_1 - x_0}, \quad u = \frac{y - y_0}{y_1 - y_0}$$

$$f_{00} = f(x_0, y_0), \quad f_{01} = f(x_0, y_1), \quad f_{10} = f(x_1, y_0), \quad f_{11} = f(x_1, y_1)$$

b. Step 1: Calculate output loading

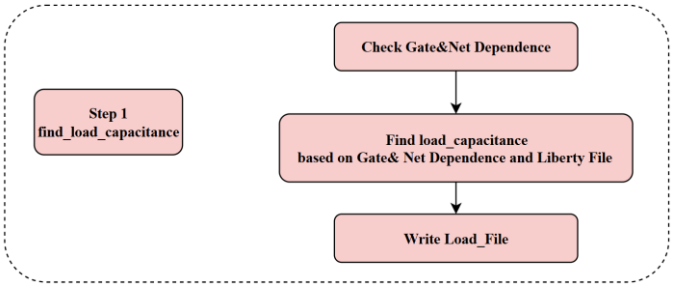
```
function find_input_capacitance(net, file):
    net_name = net.name
    input_capacitance = 0

    if net.output_gate is empty:
        return 0.03  // primary output net, assume capacitance 0.03

    for each gate in net.output_gate:
        if gate.type is "INVX1":
            input_capacitance += file.INV.A1_capacitance
        else:
            if gate.input_wire[0].name == net_name:
                if gate.type == "NANDX1":
                    input_capacitance += file.NAND.A1_capacitance
                else:
                    input_capacitance += file.NOR.A1_capacitance
            else if gate.input_wire[1].name == net_name:
                if gate.type == "NANDX1":
                    input_capacitance += file.NAND.A2_capacitance
                else:
                    input_capacitance += file.NOR.A2_capacitance

    return input_capacitance
```

首先取得 net 名稱並初始化電容值為 0。若網線沒有連接任何輸出閘門(output_gate)，直接回傳固定值 0.03，代表這是一個主要輸出端。否則，對於每一個連接的閘門，根據 gate type (ex: INVX1、NANDX1、NORX1)和網線是接在哪個輸入端 (第 0 或第 1 個 input_wire)，累加對應的電容數值(由 file 內定義，如 INV.A1、NAND.A1、NAND.A2、NOR.A1、NOR.A2 等)。最後回傳累加後的總輸入電容，這樣的計算方式是為了後續的延遲或功率分析做準備。



c. Step 2: Calculate Propagation Delay and Output Transition Time

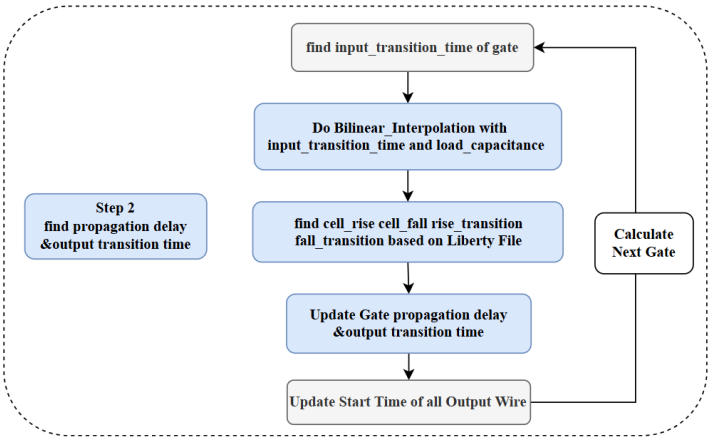
```
function topological_ordering(RTL_object, ready_gate, file) {
    initialize empty queue gate_queue

    for each gate in ready_gate {
        gate_queue.enqueue(gate)
    }

    while gate_queue is not empty {
        gate = gate_queue.dequeue()
        find_output_transition_and_delay(gate, file)

        for each output_gate in gate.output_gate {
            output_gate.in_degree -= 1
            if output_gate.in_degree == 0 {
                gate_queue.enqueue(output_gate)
            }
        }
    }
}
```

step 2 需做電路的拓撲排序，應依照拓樸順序計算 gate 的 delay 與 output transition time，一開始把所有 ready 的閘門放進隊列，接著每次取出一個閘門，計算它的輸出訊號延遲，並把它連接的下一層 gate 的 indegree 減一。如果某個 gate 的 indegree 變成零，表示它可以被處理，就加入 Queue，整個流程確保所有閘門按照訊號傳遞順序被正確處理。

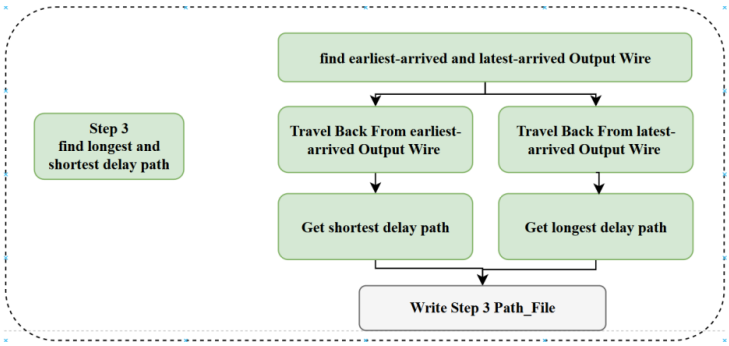


d. Step 3: Find the longest and the shortest delay from all output paths

```
function find_longest_and_shortest_path(RTL_object){
    out_wire = RTL_object.output_net
    latest_arrived_net = out_wire[0]
    earliest_arrived_net = out_wire[0]

    for i from 1 to out_wire.size - 1 {
        if(latest_arrived_net.start_time < out_wire[i].start_time){
            latest_arrived_net = out_wire[i]
        }
        if(earliest_arrived_net.start_time > out_wire[i].start_time){
            earliest_arrived_net = out_wire[i]
        }
    }
}
```

step 3 要找出所有 primary output nets 中，最晚與最早到達的 net 會遍歷 output_net，比較每個 net 的 start_time，找出最大和最小的 start_time。最晚到達的 net 對應 longest path，最早到達的 net 對應 shortest path，並可透過 find_path 追溯出完整的路徑，主要用來分析電路的延遲表現。



5. Method of My Design – Implementation Details

a. Liberty File Parser

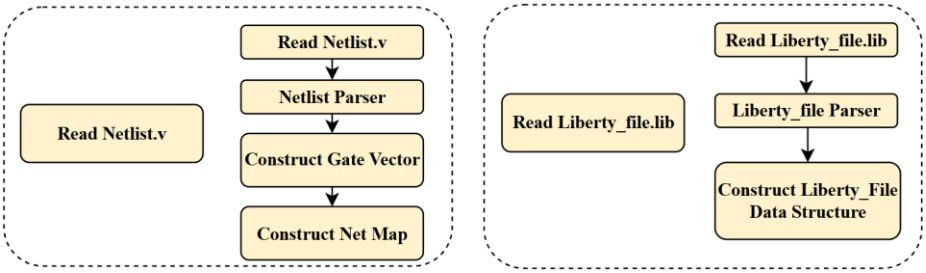
```
void build_liberty_file(struct liberty_file &file){
    vector<vector<string>> result=readBlocksFromFile("test_lib.lib");
    build_index_vec(file, result);
    build_NOR_TB (file, result[2]);
    build_INV_TB (file, result[3]);
    build_NAND_TB(file, result[4]);
}
```

build_liberty_file 負責建構一個 liberty_file 結構。首先呼叫 readBlocksFromFile 函數讀取 test_lib.lib 檔案，將其分成多個文字區塊並存到 result 中。接著，build_index_vec 會根據 result 建立索引；之後分別呼叫 build_NOR_TB、build_INV_TB 和 build_NAND_TB，根據 result 中不同位置的資料（第 2、3、4 個區塊），分別建立 NOR、INV 和 NAND 閘的查詢表（Truth Table 或 Timing Block）。

b. Netlist Parser

```
vector<string> read_RTL(const string filename,string &module_name){
    ifstream file(filename);
    vector<string> lines;
    string line;
    bool comment_mode=false;
    while (getline(file, line)){
        bool head_slash_star_bool=check_slash_star(line);
        bool end_star_slash_bool=endsWithBlockCommentEnd(line);
        if(head_slash_star_bool==true){
            comment_mode=true;
        }
        if(comment_mode==false&&check_two_slash(line)==false&&is_blank_line(line)==false){
            lines.push_back(line);
        }
        if(head_slash_star_bool==false&&end_star_slash_bool==true){
            comment_mode=false;
        }
        else if(head_slash_star_bool==true&&end_star_slash_bool==true){
            comment_mode=false;
        }
    }
    module_name=get_module_name(lines[0]);
    file.close();
    return lines;
}
```

read_RTL function 用來讀取指定檔案（filename）的內容，並將去除註解和空白行後的有效程式碼存入 vector<string> 中回傳，同時取出第一行的模組 module_name。函式會逐行讀取檔案，遇到 /*...*/ 區塊註解時會跳過，遇到 // 行註解或空白行也會忽略。透過 check_slash_star、endsWithBlockCommentEnd、check_two_slash 和 is_blank_line 等輔助函數判斷註解狀態，確保只保留真正有意義的程式碼行。最後用 get_module_name 抓取模組名。



c. Step 1: Calculate Output Loading

```
void find_load_capacitance(struct RTL_obj &RTL_object, struct liberty_file file){
    for(int i=0;i<RTL_object.gate_vec.size();i++){
        //之後可以轉成vector，因為不需要用gate名稱去查找了
        gate_node* gate=RTL_object.gate_vec[i];
        //找到該gate所有的output wire連接到的gate的種類，以及是A1 or A2
        gate->load_capacitance=find_input_capacitance(gate->output_wire[0],file);
    }
}
```

在 find_load_capacitance 函式中，目的是計算 gate 的 load capacitance，傳入的參數是 RTL_obj 結構（描述整個電路）和 liberty_file，會遍歷 RTL_object 中的每個 gate，取出

gate 的輸出線（output_wire[0]），然後呼叫 find_input_capacitance 函數，利用輸出線連接到的下一個 gate 的資訊，查出對應的輸入端電容值（A1 或 A2 腳位的電容），並將這個值累加設定到目前 gate 的 load_capacitance 成員中。

```
double find_input_capacitance(wire_node* net, struct liberty_file file){
    string net_name=net->name;
    double input_capacitance=0;
    if(net->output_gate.size()==0) return 0.03; //primary output net理論上不會貢獻電容，題目假設0.03
    for(int i=0;i<net->output_gate.size();i++){
        if(net->output_gate[i]->type=="INVX1"){
            input_capacitance+=file.INV.A1_capcitance;
        }
        else{
            if(net->output_gate[i]->input_wire[0]->name==net_name){
                if(net->output_gate[i]->type=="NANDX1")input_capacitance+=file.NAND.A1_capcitance;
                else input_capacitance+=file.NOR.A1_capcitance;
            }
            else if(net->output_gate[i]->input_wire[1]->name==net_name){
                if(net->output_gate[i]->type=="NANDX1")input_capacitance+=file.NAND.A2_capcitance;
                else input_capacitance+=file.NOR.A2_capcitance;
            }
        }
    }
    //return roundToSixDecimalPlaces(input_capacitance);
    return input_capacitance;
}
```

首先 find_load_capacitance 函式會走訪電路中所有的 gate，對每個 gate 取出它的輸出線，再用 find_input_capacitance 計算這條輸出線的總負載。find_input_capacitance 則會檢查這條線連接到哪些 gate，如果沒有連接任何 gate（代表是 primary output），就直接給一個固定的小電容值。否則，就根據連接的 gate 類型（像是 INV、NAND 或 NOR），以及線是接到 A1 還是 A2 腳位，查找對應的電容值累加起來。整個過程就是一層層把電路中訊號線的負載累積起來，為後續計算延遲和模擬做準備。

d. Step 2 : Calculate Propagation Delay and Output Transition Time

```
void find_output_transition_and_delay(gate_node* node,struct liberty_file file){
    double input_transition=find_input_transition_time_and_update_gate_start_time(node);
    double load_capacitance=node->load_capacitance;
    string gate_type=node->type;
    double cell_rise,cell_fall,rise_transition,fall_transition;
    find_result_from_TB(gate_type,input_transition,load_capacitance,file,cell_rise,
    cell_fall,rise_transition,fall_transition);

    if(cell_rise>cell_fall){
        node->propagation_delay=cell_rise;
        node->output_situation=1;
        node->output_transition_time=rise_transition;
    }
    else{
        node->propagation_delay=cell_fall;
        node->output_situation=0;
        node->output_transition_time=fall_transition;
    }
    update_start_time_of_output_wire(node);
}
```

此 function 計算每個 gate 的 output transition time 和 propagation delay。首先 find_output_transition_and_delay 會 find_input_transition_time_and_update_gate_start_time 取得目前 gate 的輸入轉換時間。接著利用 gate 的負載電容和型號，呼叫 find_result_from_TB 從標準 cell 的時序表中查出對應的 cell_rise、cell_fall、rise_transition 和 fall_transition。比較 cell_rise 和 cell_fall，若 cell_rise 較大，就設定 propagation delay 為 cell_rise，output transition time 為 rise_transition，且標記為上升；若 cell_fall 較大則反之。最後，根據計算結果更新 gate 輸出線的起始時間，為後續其他 gate 的時序推進做準備，確保每個 gate 能根

據真實負載和輸入狀態正確估算 propagation delay 和 output transition time。

```
double find_input_transition_time_and_update_gate_start_time(gate_node* node){
    if(node->input_gate.size()==0){
        node->latest_input_wire=node->input_wire[0];//對於primary input gate，取A1當latest_input_wire
        node->start_time=0;
        return 0;
    }
    else if(node->type=="INVX1"){
        node->latest_input_wire=node->input_wire[0];
        node->start_time=node->input_wire[0]->start_time+0.005;
        return node->input_wire[0]->input_gate_transition_time;
    }
    else{
        if(node->input_wire[0]->start_time+0.005 > node->input_wire[1]->start_time+0.005){
            node->latest_input_wire=node->input_wire[0];
            node->start_time=node->input_wire[0]->start_time+0.005; //取較晚到達的wire訊號作為start time
            return node->input_wire[0]->input_gate_transition_time;
        }
        else{
            node->latest_input_wire=node->input_wire[1];
            node->start_time=node->input_wire[1]->start_time+0.005; //取較晚到達的wire訊號作為start time
            return node->input_wire[1]->input_gate_transition_time;
        }
    }
}
```

此 function 在計算 gate 的 input transition time，並更新 gate 的 start time。對於 primary input 的 gate，直接取第一條輸入線 (A1) 作為 latest_input_wire，並將 start time 設為 0，輸入轉換時間也是 0。如果是 INV gate，因為只有一個輸入，直接用那條輸入線的 start time 加上一個固定延遲 0.005，再回傳該輸入線的 input transition time。其他類型的 gate (像 NAND 或 NOR) 則需要比對兩條輸入線的 start time，選擇到達較晚的那一條作為 latest_input_wire，並用它的 start time 加上 0.005 作為自己的 start time，同時回傳這條線對應的 input transition time，確保每個 gate 都從輸入線中挑出訊號到達較晚的那條，讓時序計算正確連續推進。

e. Step 3: Find the longest and the shortest delay from all output paths

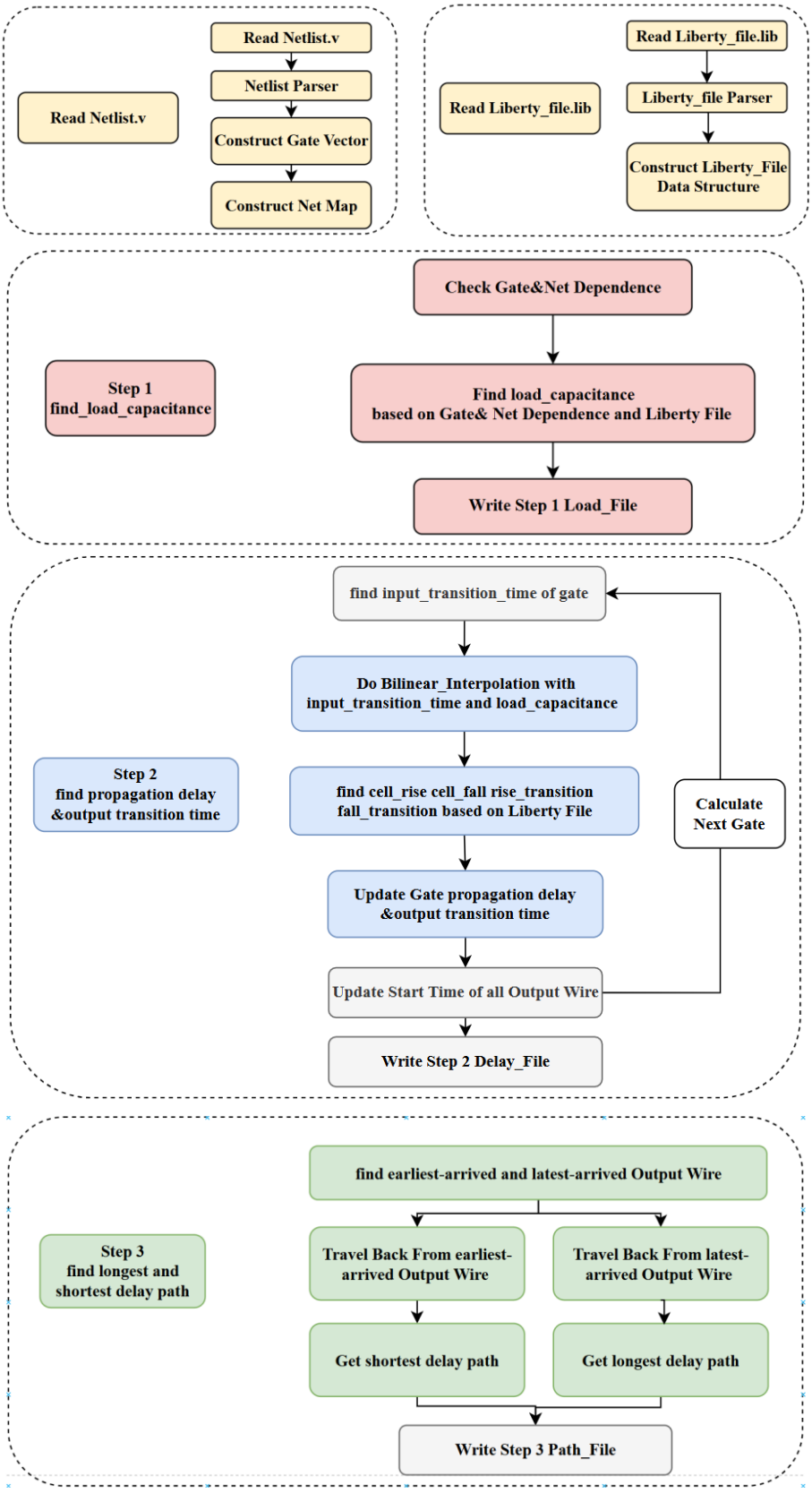
```
void find_path(wire_node* out_wire,vector<wire_node*> &path){
    wire_node* cur_wire=out_wire;
    while(cur_wire->input_gate.size()!=0){
        path.push_back(cur_wire);
        cur_wire=cur_wire->input_gate[0]->latest_input_wire;
    }
    path.push_back(cur_wire);
    cout<<endl;
}
```

此 function 從一條 output wire 開始，沿著電路中每個 gate 的 latest input wire 一直往前追溯，直到遇到 primary input（也就是沒有 input gate 的地方）。沿途經過的 wire 都會依序存進 path 這個 vector，形成一條從輸出回溯到輸入的完整路徑。

```
void find_longest_and_shortest_path(struct RTL_obj &RTL_object){
    vector<wire_node*>out_wire=RTL_object.output_net;
    wire_node* latest_arrived_net=out_wire[0];
    wire_node* earliest_arrived_net=out_wire[0];
    for(int i=1;i<out_wire.size();i++){
        if(latest_arrived_net->start_time < out_wire[i]->start_time){
            latest_arrived_net= out_wire[i];
        }
        if(earliest_arrived_net->start_time > out_wire[i]->start_time){
            earliest_arrived_net= out_wire[i];
        }
    }
}
```

此 function 負責在所有 output nets 中，找到信號最晚到達（start time 最大）和最早到達（start time 最小）的輸出線。function 會遍歷 RTL_object 裡的所有輸出線 (output wire)，分別比較它們的 start time，最後將最晚到達的線存到 latest_arrived_net，最早到達的 wire 記錄到 earliest_arrived_net。

6. Method of My Design - Flow Chart of My Program



7. Test Cases Results

a. Public Case 1

正確運行，通過 checker 測試

```
[s110501521@ncuee 110501521_PA3]$ ./PA3_checker c17.v test_lib.lib Load_110501521_c17.txt Delay_110501521_c17.txt Path
_110501521_c17.txt
Step 1 Correct!!!
Step 2 Correct!!!
Step 3-1 Correct!!!
Step 3-2 Correct!!!
Score: 8
```

b. Public Case 2

正確運行，通過 checker 測試

```
[s110501521@ncuee 110501521_PA3]$ ./PA3_checker example.v test_lib.lib Load_110501521_example.txt Delay_110501521_exam
ple.txt Path_110501521_example.txt
Step 1 Correct!!!
Step 2 Correct!!!
Step 3-1 Correct!!!
Step 3-2 Correct!!!
Score: 8
```

c. My Self-Generated Case 1: 此 testcase 是把 c17.v 加上各種註解後進行測試

```
module c17_v2 (N1, N2, N3, N6, N7, N22, N23);
    input N1, N2, N3, N6, N7;
    output N22, N23;
    wire n8, n9, n10, n11, n12;
    /*wqeqwe 879n qweq we */
    /* so won't you stay with
    oipi   poipi   poiopi
        me cutaqweqw */
    NOR2X1 U8 (.A1(n8), .A2(n9), .ZN(N23));
    NOR2X1 U9 (.A1(N2), .A2(N7), .ZN(n9));
    /*wqeqwe 879n qweq we */
    /* so won't you stay with
    oipi   poipi   poiopi
        me cutaqweqw */
    INVX1 U10 (.I(n10), .ZN(n8));
    /*wqeqwe 879n qweq we */
    /* so won't you stay with
    oipi   poipi   poiopi
        me cutaqweqw */
    NANDX1 U11 (.A1(n11), .A2(n12), .ZN(N22));
    NANDX1 U12 (.A1(N2), .A2(n10), .ZN(n12));
    NANDX1 U13 (.A1(N6), .A2(N3), .ZN(n10));
    NANDX1 U14 (.A1(N1), .A2(N3), .ZN(n11));
endmodule
```

通過 checker 測試:

```
[s110501521@ncuee 110501521_PA3]$ ./PA3_checker  c17_v2.v test_lib.lib Load_110501521_c17_v2.txt Delay_110501521_c17_v
2.txt  Path_110501521_c17_v2.txt
Step 1 Correct!!!
Step 2 Correct!!!
Step 3-1 Correct!!!
Step 3-2 Correct!!!
Score: 8
```

d. My Self-Generated Case 2: 此 testcase 共有 48 個 net 以及 37 個 gate

```
module c17_v3 (N1, N2, N3, N4, N5, N6, N7, N8, N9, N10,N30, N31, N32, N33, N34, N
    input N1, N2, N3, N4, N5, N6, N7, N8, N9, N10;
    output N30, N31, N32, N33, N34, N35, N36, N37;

/*wqeqwe 879n qweq we */
 /* so won't you stay with
 oipi   poipi   poiopi
    me cutaqweqw */
    wire n11, n12, n13, n14, n15, n16, n17, n18, n19, n20;
    wire n21, n22, n23, n24, n25, n26, n27, n28, n29, n38;
    wire n39, n40, n41, n42, n43, n44, n45, n46, n47, n48;

    NANDX1 U1 (.A1(N1), .A2(N2), .ZN(n11));
    NOR2X1 U2 (.A1(N3), .A2(N4), .ZN(n12));
    INVX1  U3 (.I(N5), .ZN(n13));
    NANDX1 U4 (.A1(N6), .A2(N7), .ZN(n14));
    NOR2X1 U5 (.A1(N8), .A2(N9), .ZN(n15));
    INVX1  U6 (.I(N10), .ZN(n16));
/*wqeqwe 879n qweq we */
 /* so won't you stay with
 oipi   poipi   poiopi
    me cutaqweqw */

    NANDX1 U7 (.A1(n11), .A2(n12), .ZN(n17));
    NOR2X1 U8 (.A1(n13), .A2(n14), .ZN(n18));
    INVX1  U9 (.I(n15), .ZN(n19));
    NANDX1 U10 (.A1(n16), .A2(n17), .ZN(n20));

    NOR2X1 U11 (.A1(n18), .A2(n19), .ZN(n21));
    NANDX1 U12 (.A1(n20), .A2(n21), .ZN(n22));
    INVX1  U13 (.I(n22), .ZN(n23));

    NOR2X1 U14 (.A1(n17), .A2(n18), .ZN(n24));
    NANDX1 U15 (.A1(n19), .A2(n24), .ZN(n25));
    INVX1  U16 (.I(n20), .ZN(n26));

    NANDX1 U17 (.A1(n25), .A2(n26), .ZN(n27));
    NOR2X1 U18 (.A1(n23), .A2(n27), .ZN(n28));
    NANDX1 U19 (.A1(n28), .A2(n21), .ZN(n29));

    INVX1  U20 (.I(n29), .ZN(N30));

    NOR2X1 U21 (.A1(n28), .A2(n26), .ZN(n38));
    NANDX1 U22 (.A1(n38), .A2(n25), .ZN(N31));
```

```
    INVX1  U23 (.I(n38), .ZN(n39));
    NOR2X1 U24 (.A1(n39), .A2(n23), .ZN(n40));

    NANDX1 U25 (.A1(n40), .A2(n27), .ZN(N32));

    NOR2X1 U26 (.A1(n40), .A2(n19), .ZN(n41));
    NANDX1 U27 (.A1(n41), .A2(n12), .ZN(N33));

    INVX1  U28 (.I(n41), .ZN(n42));
    NOR2X1 U29 (.A1(n42), .A2(n18), .ZN(n43));

    NANDX1 U30 (.A1(n43), .A2(n11), .ZN(N34));

    NOR2X1 U31 (.A1(n43), .A2(n14), .ZN(n44));
    NANDX1 U32 (.A1(n44), .A2(n15), .ZN(N35));

    INVX1  U33 (.I(n44), .ZN(n45));
    NOR2X1 U34 (.A1(n45), .A2(n16), .ZN(n46));

    NANDX1 U35 (.A1(n46), .A2(n13), .ZN(N36));

    NOR2X1 U36 (.A1(n46), .A2(n14), .ZN(n47));
    NANDX1 U37 (.A1(n47), .A2(n19), .ZN(N37));

endmodule
```

通過 checker 測試:

```
[s110501521@ncuee 110501521_PA3]$ ./PA3_checker  c17_v3.v test_lib.lib Load_110501521_c17_v3.txt Delay_110501521_c17_v
3.txt Path_110501521_c17_v3.txt
Step 1 Correct!!!
Step 2 Correct!!!
Step 3-1 Correct!!!
Step 3-2 Correct!!!
Score: 8
```

e. My Self-Generated Case 3

此 testcase 共有 99 個 net，62 個 gate

```
module c17_v6 (
    N1, N2, N3, N4, N5, N6, N7, N8, N9, N10,
    N11, N12, N13, N14, N15, N16, N17, N18, N19, N20,
    N21, N22, N23, N24, N25, N26, N27, N28, N29, N30,
    N90, N91, N92, N93, N94, N95, N96, N97, N98, N99
);
    input N1, N2, N3, N4, N5, N6, N7, N8, N9, N10;
    input N11, N12, N13, N14, N15, N16, N17, N18, N19, N20;
    input N21, N22, N23, N24, N25, N26, N27, N28, N29, N30;
    output N90, N91, N92, N93, N94, N95, N96, N97, N98, N99;

    wire n31, n32, n33, n34, n35, n36, n37, n38, n39, n40;
    wire n41, n42, n43, n44, n45, n46, n47, n48, n49, n50;
    wire n51, n52, n53, n54, n55, n56, n57, n58, n59, n60;
    wire n61, n62, n63, n64, n65, n66, n67, n68, n69, n70;
    wire n71, n72, n73, n74, n75, n76, n77, n78, n79, n80;
    wire n81, n82, n83, n84, n85, n86, n87, n88, n89;

    // First Layer
    NANDX1 U1 (.A1(N1), .A2(N2), .ZN(n31));
    NANDX1 U2 (.A1(N3), .A2(N4), .ZN(n32));
    NANDX1 U3 (.A1(N5), .A2(N6), .ZN(n33));
    NOR2X1  U4 (.A1(N7), .A2(N8), .ZN(n34));
    NOR2X1  U5 (.A1(N9), .A2(N10), .ZN(n35));
    INVX1   U6 (.I(N11), .ZN(n36));
    INVX1   U7 (.I(N12), .ZN(n37));
    NANDX1 U8 (.A1(N13), .A2(N14), .ZN(n38));
    NOR2X1  U9 (.A1(N15), .A2(N16), .ZN(n39));
    NANDX1 U10 (.A1(N17), .A2(N18), .ZN(n40));

    // Second Layer
    NOR2X1  U11 (.A1(n31), .A2(n32), .ZN(n41));
    NANDX1 U12 (.A1(n33), .A2(n34), .ZN(n42));
    NOR2X1  U13 (.A1(n35), .A2(n36), .ZN(n43));
    NANDX1 U14 (.A1(n37), .A2(n38), .ZN(n44));
    NOR2X1  U15 (.A1(n39), .A2(n40), .ZN(n45));

    // Third Layer
    NANDX1 U16 (.A1(n41), .A2(n42), .ZN(n46));
    NOR2X1  U17 (.A1(n43), .A2(n44), .ZN(n47));
    INVX1   U18 (.I(n45), .ZN(n48));
```

```
    NANDX1 U30 (.A1(n55), .A2(n56), .ZN(n60));

    // Sixth Layer
    NANDX1 U31 (.A1(n57), .A2(n58), .ZN(n61));
    NOR2X1  U32 (.A1(n59), .A2(n60), .ZN(n62));

    INVX1   U33 (.I(n61), .ZN(n63));
    NANDX1 U34 (.A1(n63), .A2(n62), .ZN(n64));
    INVX1   U35 (.I(n64), .ZN(n65));

    // Output Generation
    NOR2X1  U36 (.A1(n65), .A2(N30), .ZN(n66));
    NANDX1 U37 (.A1(n66), .A2(n50), .ZN(n67));
    INVX1   U38 (.I(n67), .ZN(n68));
    NOR2X1  U39 (.A1(n68), .A2(n49), .ZN(n69));
    NANDX1 U40 (.A1(n69), .A2(n46), .ZN(n70));
    INVX1   U41 (.I(n70), .ZN(n71));
    NOR2X1  U42 (.A1(n71), .A2(n43), .ZN(n72));
    NANDX1 U43 (.A1(n72), .A2(n41), .ZN(n73));
    INVX1   U44 (.I(n73), .ZN(n74));
    NOR2X1  U45 (.A1(n74), .A2(n33), .ZN(n75));
    NANDX1 U46 (.A1(n75), .A2(n31), .ZN(n76));
    INVX1   U47 (.I(n76), .ZN(n77));
    NOR2X1  U48 (.A1(n77), .A2(N5), .ZN(n78));
    NANDX1 U49 (.A1(n78), .A2(N3), .ZN(n79));
    INVX1   U50 (.I(n79), .ZN(n80));
    NOR2X1  U51 (.A1(n80), .A2(N1), .ZN(n81));
    NANDX1 U52 (.A1(n81), .A2(N2), .ZN(n82));

    // Final Outputs
    INVX1   U53 (.I(n82), .ZN(N90));
    INVX1   U54 (.I(n81), .ZN(N91));
    INVX1   U55 (.I(n80), .ZN(N92));
    INVX1   U56 (.I(n79), .ZN(N93));
    INVX1   U57 (.I(n78), .ZN(N94));
    INVX1   U58 (.I(n77), .ZN(N95));
    INVX1   U59 (.I(n76), .ZN(N96));
    INVX1   U60 (.I(n75), .ZN(N97));
    INVX1   U61 (.I(n74), .ZN(N98));
    INVX1   U62 (.I(n73), .ZN(N99));

endmodule
```

通過 checker 測試:

```
[s110501521@ncuee 110501521_PA3]$ ./PA3_checker  c17_v6.v test_lib.lib Load_110501521_c17_v6.txt Delay_110501521_c17_v
6.txt Path_110501521_c17_v6.txt
Step 1 Correct!!!
Step 2 Correct!!!
Step 3-1 Correct!!!
Step 3-2 Correct!!!
Score: 8
```

8. Reference

a. Gattiker, A., Nassif, S., Dinakar, R., & Long, C. (2001, March). Timing yield estimation from static timing analysis. In *Proceedings of the IEEE 2001. 2nd International Symposium on Quality Electronic Design* (pp. 437-442). IEEE.

b. Forzan, Cristiano, and Davide Pandini. "Statistical static timing analysis: A survey." *Integration* 42.3 (2009): 409-435.

c. Jourdan, M., & Maraninchi, F. (1995). Static timing analysis of real-time systems. *ACM SIGPLAN Notices*, *30*(11), 79-87.

d. Devgan, Anirudh, and Chandramouli Kashyap. "Block-based static timing analysis with uncertainty." *ICCAD-2003. International Conference on Computer Aided Design (IEEE Cat. No. 03CH37486)*. IEEE, 2003.